TUTORIAL 1 • AGENTS, HANDS ON • LINE BY LINE

Five agent architectures, one filthy room

A complete walkthrough of the vacuum-world notebook — **every line of code explained**, every output shown as it actually ran, and full worked solutions to all four exercises. Read it top to bottom and you will know not just what each agent does, but why the architectures stop being interchangeable the moment the world gets harder.

BASED ON	CODE	OUTPUTS
Tutorial_1_Agents_Hands_On.ipynb	Python 3.11, no dependencies	Executed, not transcribed

COVERS

9 cells + 4 solutions

---

00 The one idea to hold on to

Everything in this notebook is a variation on a single loop. The agent *senses*, *decides*, and *acts*; the world changes; it senses again.

An **agent** is anything that perceives its environment through **sensors** and acts on it through **actuators**. What it does is defined by an **agent function** that maps a percept sequence to an action. That function is an abstraction; the **agent program** is the concrete code that implements it. The five architectures in this tutorial are five different ways of writing that program.

> THE SPLIT THAT MATTERS
>
> The notebook keeps the **environment** and the **agent** in separate objects that share nothing but a percept and an action string. The environment knows the whole world; the

> agent knows only what `percept()` hands it. Every interesting result later comes from that gap.

What each architecture adds

ARCHITECTURE	WHAT IT ADDS	THE QUESTION IT CAN ANSWER
Simple reflex	Nothing — rules on the current percept only	“What is under me right now?”
Model-based reflex	Internal state: memory of what it has seen	“What is the world like, including where I am not?”
Goal-based	An explicit goal and a plan to reach it	“What sequence gets me to the state I want?”
Utility-based	A scoring function over competing outcomes	“Which of these imperfect options is best?”
Learning	A feedback loop that edits its own policy	“What worked last time?”

They are cumulative in capability but not in cost. The whole payoff of the exercises at the end is discovering that more capability is not automatically better — each addition brings its own failure mode.

01 The environment (the world the agent lives in)

This is the longest cell in the notebook and the most important. Read it as a contract: it defines exactly what the agent is allowed to know and exactly what it is allowed to do.

Cell 1 — the vacuum world

```python
import random
from copy import deepcopy

LOCATIONS = ["A", "B"]

class VacuumEnvironment:
    def __init__(self, dirt_prob=0.5, seed=42):
        rng = random.Random(seed)
        self.status = {loc: ("Dirty" if rng.random() < dirt_prob else "Clean") fo
        self.agent_location = rng.choice(LOCATIONS)
        self.performance = 0       # scorecard: +10 per successful clean, -1 per
        self.history = []               # (location, action, world_state) trace f

    def percept(self):
        '''Sensors: the agent perceives only its current location and its status
        return (self.agent_location, self.status[self.agent_location])

    def execute(self, action):
        '''Actuators: apply the chosen action to the environment.'''
        loc = self.agent_location
        if action == "Suck":
            if self.status[loc] == "Dirty":
                self.status[loc] = "Clean"
                self.performance += 10
        elif action == "Left":
            self.agent_location = "A"
            self.performance -= 1
        elif action == "Right":
            self.agent_location = "B"
            self.performance -= 1
        elif action == "NoOp":
            pass
        else:
            raise ValueError(f"Unknown action: {action}")

        self.history.append((loc, action, deepcopy(self.status)))
        return self.percept()

    def is_clean(self):
        return all(v == "Clean" for v in self.status.values())


# Quick sanity check
env = VacuumEnvironment(seed=1)
```

```
print("Initial world state:", env.status)
print("Initial percept:", env.percept())
```

L1	The only import the core tutorial needs. `random` generates the starting world — and later, the learning agent's exploration.
L2	`deepcopy` is imported for one job, on line 36. It is what makes the history a genuine *trace* instead of the same dictionary repeated.
L4	The world is two squares, `A` and `B`. A module-level constant so both the environment and the agents refer to the same world. Exercise 3 changes this line and discovers how much depends on it.
L6	The environment is its own class. The agent is not a method on it — that separation is the whole design.
L7	`dirt_prob` is the chance each square starts dirty. `seed` makes a run reproducible, which is what lets you compare architectures fairly.
L8	A *private* random generator, not the global `random` module. So the world's layout cannot be shifted by anything else in the notebook calling `random` — which matters, because the learning agent does exactly that.
L9	A dict comprehension building `{'A': 'Dirty'|'Clean', 'B': ...}`. Each square is dirty with probability `dirt_prob`. This dictionary *is* the world.
L10	The agent's starting square, also drawn from the seeded generator.
L11	The scorecard, starting at zero. The comment states the whole performance measure: **+10 per successful clean, −1 per move**. That ratio is what makes travelling worthwhile — and Exercise 4 is about what happens when you change it.
L12	An empty list that will collect `(location, action, world_state)` triples so you can inspect behaviour afterwards.
L14–16	**The sensors.** The single most consequential method here. It returns a tuple of the agent's *current location* and the status of *that square only*. It cannot see the other square. Everything the agent does not know, it does not know because of this one line.
L18–19	**The actuators.** Takes an action string and applies its effect to the world.
L20	Cache the location *before* the action potentially moves the agent. Line 36 needs the square the action was taken *from*.

L21–24	`Suck` only scores when the square was actually dirty. Sucking a clean square is silently free: no reward, and no penalty either. Harmless here — but that free no-op is exactly the loophole Exercise 1 catches the utility agent exploiting.
L25–27	`Left` sets the location to `"A"` — note this is **absolute, not relative**. In a two-square world “go left” and “go to A” are the same thing. That coincidence hides a real assumption, and Exercises 2 and 3 both turn on it. Cost: −1.
L28–30	`Right` is the mirror image: go to `B`, pay −1.
L31–32	`NoOp` changes nothing and costs nothing. It is the only free action, which makes it the baseline every utility comparison is measured against.
L33–34	Anything else is a programming error, raised loudly. A cheap contract check that stops a typo in an agent from silently becoming a no-op.
L36	Append the step to the trace. The `deepcopy` matters: `self.status` is mutated in place, so storing it directly would leave every entry pointing at the same dict and the trace would show the final state six times over.
L37	Return the fresh percept as a convenience. The runner ignores it and calls `percept()` itself, so this is belt-and-braces.
L39–40	A god's-eye check on whether the whole world is clean. Note carefully: **this is a method on the environment, not the agent.** The agent can never call it — it would be cheating. Knowing when to stop is something each architecture must earn for itself.
L43–46	A sanity check. Build a world with `seed=1`, print the true state, then print what the agent would actually perceive.

OUTPUT

```
Initial world state: {'A': 'Dirty', 'B': 'Clean'}
Initial percept: ('A', 'Dirty')
```

Look at those two output lines together. The world is 

— but the percept is just `('A', 'Dirty')`. The fact that `B` is already clean exists in the environment and is invisible to the agent. Hold that thought; it is the entire plot.

02 PEAS: specifying the task before you code it

PEAS — Performance, Environment, Actuators, Sensors — is a checklist for pinning down a task precisely enough to build for it. Here it is written out as a plain dictionary.

Cell 2 — the task specification

```
1  vacuum_world_peas = {
2      "Performance": "Reward for clean squares, small penalty per move",
3      "Environment": "Two locations (A, B), each Dirty or Clean",
4      "Actuators": "Suck, Left, Right, NoOp",
5      "Sensors": "Current location, dirt status of current location",
6  }
7
8  for k, v in vacuum_world_peas.items():
9      print(f"{k:12s}: {v}")
```

L1–6	A plain dictionary, one key per PEAS letter. There is no clever machinery here on purpose: the value is in being forced to *write the four answers down* before writing agent code.
L2	**Performance**: reward for clean squares, small penalty per move. Note that this is a description of lines 11, 24, 27 and 30 of the previous cell — the spec and the code have to agree, and nothing enforces it but you.
L3	**Environment**: two locations, each dirty or clean. Small, static, fully observable at each square, and episodic. Every one of those four properties is deliberately broken by one of the exercises.
L4	**Actuators**: the four legal action strings, matching the branches of `execute()` exactly.
L5	**Sensors**: current location and dirt status *of the current location*. The phrasing carries the limitation.
L8–9	Print each pair. `f"{k:12s}"` left-pads the key to 12 characters so the colons line up into a column.

OUTPUT

```
Performance : Reward for clean squares, small penalty per move
Environment : Two locations (A, B), each Dirty or Clean
Actuators   : Suck, Left, Right, NoOp
Sensors     : Current location, dirt status of current location
```

WHY THIS CELL EARNS ITS PLACE

Every failure in the exercise section is a PEAS assumption that stopped being true. Dirt that reappears breaks *Environment*. Hiding the location breaks *Sensors*. A deadline breaks *Performance*. Writing PEAS down is what makes those failures legible instead of mysterious.

03 The run loop (the harness everything is scored by)

Seven lines, and it is the sense–decide–act cycle in its plainest possible form. Every score in this tutorial comes out of this function.

Cell 4 — the harness

```python
def run(agent_program, steps=6, seed=42):
    env = VacuumEnvironment(seed=seed)
    for _ in range(steps):
        p = env.percept()
        a = agent_program(p)
        env.execute(a)
    return env.performance, env.is_clean(), env.history
```

L1	Takes an *agent program* — a function from percept to action — plus a step budget and a seed. Because the agent is just a function, every architecture can be scored by the identical harness.
L2	A fresh environment per run. Passing the same seed to every agent guarantees they all face the same starting world, which is what makes the comparison table meaningful.
L3	Run for a fixed number of steps. `_` signals the counter is unused. Note there is **no termination check** — the loop does not stop when the world is clean. That is deliberate: it means idling has to be a choice the agent makes, not a favour the harness does for it.
L4	**Sense.** Ask the environment what the agent can currently perceive.
L5	**Decide.** Hand that percept to the agent program and receive an action string. This line is the entire agent interface.
L6	**Act.** Apply the action to the environment, which updates the world, the score and the trace.
L7	Return final score, whether the world ended up clean, and the full trace.

A DETAIL WORTH NOTICING

Six steps is a tight budget for a world that needs at most one move and two sucks. That tightness is why `NoOp` is worth points: an agent that keeps pacing after the job is done burns −1 per step, and there are only six steps to spend.

04 Architecture 1 — Simple reflex

Condition–action rules on the current percept. No memory of anything.

Cell 3 — simple reflex agent

```python
def simple_reflex_agent():
    '''Rule: if the current square is dirty, clean it; otherwise move to the othe
    def program(percept):
        location, status = percept
        if status == "Dirty":
            return "Suck"
        elif location == "A":
            return "Right"
        else:
            return "Left"
    return program
```

L1	A *factory*: calling `simple_reflex_agent()` returns a fresh program. Every architecture in the notebook follows this shape, so the comparison loop can call `factory()` uniformly and each agent starts with clean internal state.
L2	The docstring is the agent function in English: dirty → clean it, otherwise move.
L3	The inner function is the actual agent program. Its only input is `percept` — no history, no world access.
L4	Unpack the tuple from `percept()` into the two things the agent knows.
L5–6	Rule one: if the square under me is dirty, suck. Immediate and always correct.
L7–8	Rule two: a clean square at `A` means go right.
L9–10	Rule three: otherwise (clean, at `B`) go left.
L11	Return the program. Nothing is captured in the closure — **there is no state to capture.** That absence is the definition of this architecture.

Watching it run

The trace cell runs the agent and prints one line per step.

```
Cell 5 — trace, seed=3
```

```python
_, _, trace = run(simple_reflex_agent(), seed=3)
for step, (loc, action, state) in enumerate(trace, 1):
    print(f"Step {step}: at {loc} -> action={action:6s} -> world={state}")
```

L1	Run the agent and keep only the trace; `_, _,` discards score and cleanliness. Note `seed=3` — a different world from the default.
L2	`enumerate(trace, 1)` numbers the steps from 1, and each entry unpacks into the triple stored on line 36 of the environment.
L3	`{action:6s}` pads the action to six characters so the arrows line up into a readable column.

OUTPUT

```
Step 1: at B -> action=Left   -> world={'A': 'Dirty', 'B': 'Clean'}
Step 2: at A -> action=Suck   -> world={'A': 'Clean', 'B': 'Clean'}
Step 3: at A -> action=Right  -> world={'A': 'Clean', 'B': 'Clean'}
Step 4: at B -> action=Left   -> world={'A': 'Clean', 'B': 'Clean'}
Step 5: at A -> action=Right  -> world={'A': 'Clean', 'B': 'Clean'}
Step 6: at B -> action=Left   -> world={'A': 'Clean', 'B': 'Clean'}
```

Step by step, with the world drawn after each action:

1. Starts at **B**, which is clean, so the rule says go left.

2. Now at **A**, which is dirty → `Suck`. The world is now spotless.

3. At **A**, clean → the rule says go right. Cost: −1.
4. At **B**, clean → go left. Cost: −1.
5. …and it paces back and forth until the step budget runs out.

THE FAILURE

From step 3 onward this agent is *doing damage* — four wasted moves, −4 points, in a world that has been clean since step 2. It cannot stop, and the reason is precise: stopping requires knowing that *both* squares are clean, and it can only ever see one. Final score: **+10 − 5 = 5**.

05 Architecture 2 — Model-based reflex

The same reflex rules, plus one dictionary of memory. That dictionary is worth four points.

Cell 6 — model-based reflex agent

```
1  def model_based_reflex_agent():
2      model = {"A": None, "B": None}   # internal state: what we believe about eacl
3
4      def program(percept):
5          location, status = percept
6          model[location] = status              # update our internal model with wl
7          if status == "Dirty":
8              return "Suck"
9          if model["A"] == "Clean" and model["B"] == "Clean":
10             return "NoOp"                          # we believe the job is done -
11         return "Right" if location == "A" else "Left"
12     return program
```

L1	Same factory shape as before.
L2	**The internal state.** One dict holding what the agent believes about each square, starting at `None` for "never looked". Because it lives in the enclosing function, it persists across calls to `program` — this is a closure doing the work an object would do.
L4–5	The program and the same unpacking as before.
L6	**Update the model.** Record what was just observed, keyed by location. The `None` for the other square is what stops the agent claiming knowledge it does not have.
L7–8	Unchanged reflex rule: dirt under me gets sucked immediately.
L9–10	**The new capability.** If the model says both squares are clean, emit `NoOp` and stop burning move penalties. This condition is literally inexpressible for the previous agent — it mentions a square the agent is not standing on.
L11	Otherwise fall back to the reflex move. Reaching this line means at least one square is still `None` or `Dirty`, so there is a reason to travel.
L12	Return the program — this time *with* `model` captured in its closure.

Cell 7 — trace, same seed=3

```
_, _, trace = run(model_based_reflex_agent(), seed=3)
for step, (loc, action, state) in enumerate(trace, 1):
    print(f"Step {step}: at {loc} -> action={action:6s} -> world={state}")
```

L1–3 Identical to the previous trace cell, with the agent swapped. Same world, same seed, same harness — so any difference in output is caused by architecture alone. That is the experiment.

OUTPUT

```
Step 1: at B -> action=Left   -> world={'A': 'Dirty', 'B': 'Clean'}
Step 2: at A -> action=Suck   -> world={'A': 'Clean', 'B': 'Clean'}
Step 3: at A -> action=NoOp   -> world={'A': 'Clean', 'B': 'Clean'}
Step 4: at A -> action=NoOp   -> world={'A': 'Clean', 'B': 'Clean'}
Step 5: at A -> action=NoOp   -> world={'A': 'Clean', 'B': 'Clean'}
Step 6: at A -> action=NoOp   -> world={'A': 'Clean', 'B': 'Clean'}
```

The first two steps are identical to the reflex agent's. Step 3 is where they part:

STEP	SIMPLE REFLEX	MODEL-BASED	WHY
1	`Left`	`Left`	Both see a clean B, both move
2	`Suck`	`Suck`	Both see dirt at A
3	`Right`	**`NoOp`**	The model now holds A=Clean *and* B=Clean, from step 1
4–6	pacing	`NoOp`	−3 versus 0

THE LESSON

5 → 9 points, from one dictionary. The model-based agent is not smarter about cleaning — it cleans exactly the same dirt on the same step. It is smarter about *stopping*. Memory did not buy better actions, it bought the ability to know the job was finished. Keep that distinction; Exercise 1 is about what happens when that belief goes stale.

06 Architecture 3 — Goal-based

Instead of firing rules, the agent holds an explicit goal — “both squares clean” — and forms a small plan to reach it.

Cell 8 — goal-based agent

```
def goal_based_agent():
    goal_reached_plan = []                # a tiny plan: where to go next once th

    def program(percept):
        location, status = percept
        nonlocal goal_reached_plan
        if status == "Dirty":
            return "Suck"                       # always satisfy the immediate su
        if not goal_reached_plan:
            move = "Right" if location == "A" else "Left"
            goal_reached_plan = [move]           # plan: go check the other squar
        return goal_reached_plan.pop(0)
    return program
```

L1	Same factory shape.
L2	The plan: a list of queued actions. Empty means “no plan right now, make one”. In a real goal-based agent this is where a search algorithm's output would land; here the world is small enough that the plan is one move long.
L4–5	The program and the usual unpacking.
L6	`nonlocal` lets line 11 *rebind* `goal_reached_plan` to a new list. Without it, that assignment would create a fresh local variable and the plan would silently vanish between calls. (Line 6 is not needed for the `.pop()` on line 12, which mutates rather than rebinds — it is needed specifically because of line 11.)
L7–8	Dirt under the agent is an immediate sub-goal, satisfied before any planning happens.
L9–11	If there is no plan, make one: the only useful thing to do on a clean square is go check the other one.
L12	Pop the front of the plan and execute it. With a one-step plan this is the same as returning the move — but the *shape* is the point: decide a sequence, then follow it.
L13	Return the program, with the plan captured in the closure.

READ THE CODE, NOT THE LABEL

This agent has a goal in its name but nowhere in its logic. There is no check for “is the goal satisfied?” and therefore **no `NoOp` branch anywhere**. It always returns `Suck` or a move, so it paces forever exactly like the simple reflex agent — and scores exactly the same, **5**. Watch for this in the comparison table; it is the tutorial's most useful trap. The architecture is only as good as the goal test you actually write.

07 Architecture 4 — Utility-based

A goal is binary: reached or not. A utility function is a number, so it can *trade off* competing concerns — here, cleanliness against a battery budget.

Cell 9 — utility-based agent

```python
def utility_based_agent(battery_start=6):
    battery = {"level": battery_start}

    def utility(action, status, battery_level):
        u = 0
        if action == "Suck" and status == "Dirty":
            u += 10
        if action in ("Left", "Right"):
            u -= 2                                          # moving costs energ
        if battery_level <= 2 and action in ("Left", "Right"):
            u -= 5                                          # heavily discourage
        return u

    def program(percept):
        location, status = percept
        if battery["level"] <= 0:
            return "NoOp"                               # out of battery — must stop
        candidates = ["Suck", "Left", "Right", "NoOp"]
        action = max(candidates, key=lambda a: utility(a, status, battery["level"
        if action in ("Left", "Right", "Suck"):
            battery["level"] -= 1
        return action
    return program
```

L1	The battery budget is a parameter, so you can experiment with scarcity.
L2	The battery is wrapped in a dict rather than a bare number for the same reason as before: rebinding a plain `int` inside the nested function would need `nonlocal`, while mutating a dict entry does not.
L4	**The utility function.** It scores a hypothetical action — nothing is applied to the world. This is the key move of the architecture: the agent can compare options it will not take.
L5	Every action starts at zero utility.
L6–7	`+10` for sucking actual dirt — deliberately mirroring the environment's own reward on line 24. Note this only fires when the square *is* dirty; `Suck` on a clean square scores 0.
L8–9	`-2` for any move. Note this is *harsher* than the environment's real −1 — the agent's preferences do not have to match the true performance measure, and the mismatch has consequences.
L10–11	A further −5 for moving on a nearly flat battery. This is the actual trade-off: at full charge a move is a minor cost, at low charge it is close to unthinkable.
L12	Return the score.
L14–15	The program proper.
L16–17	A hard constraint that overrides the arithmetic: no battery, no action. Some limits should not be negotiable in utility terms.
L18	All four legal actions become candidates — including `NoOp`, which always scores exactly 0 and therefore acts as the **do-nothing baseline**. Any action scoring below 0 is worse than idling.
L19	`max` with a `key` scores every candidate and picks the best. On a tie Python keeps the *first* maximum in list order, so the order on line 18 is a silent tie-breaker — `Suck` wins ties over `NoOp`.
L20–21	Any real action drains one unit of battery; `NoOp` is free.
L22–23	Return the action, then the program.

> WHERE THIS SCORES 10

In the standard six-step run this is the joint best agent. On a clean square, `Suck` and `NoOp` both score 0 while moving scores −2, so the tie on line 19 goes to `Suck` — a harmless no-op in the environment. It cleans what is under it and never wastes a move.

…AND WHERE IT IS QUIETLY BROKEN

Read that again: **moving is never worth it.** A move always scores −2, and the utility function has no term for “there might be dirt over there.” This agent only ever scores 10 because it happens to start on the dirty square. Exercise 1 puts it in a world where it has to travel, and it parks on one square forever. The bug is invisible in the tutorial's own test.

08 Architecture 5 — Learning

A learning element that edits the agent's own policy from observed reward — a toy version of the feedback loop behind RLHF.

Cell 10 — learning agent

```python
def learning_agent():
    q = {}                                  # (state, action) -> learned value
    last = {"state": None, "action": None}

    def program(percept):
        state = percept
        actions = ["Suck", "Left", "Right"]
        for a in actions:
            q.setdefault((state, a), 0.0)

        # Learning element: update the value of the last (state, action) using th
        if last["state"] is not None:
            reward = 10 if state[1] == "Clean" and last["action"] == "Suck" else
            key = (last["state"], last["action"])
            q[key] += 0.5 * (reward - q[key])       # simple running-average upda

        # Performance element: mostly exploit the best known action, occasionally
        if random.random() < 0.1:
            action = random.choice(actions)
        else:
            action = max(actions, key=lambda a: q[(state, a)])

        last["state"], last["action"] = state, action
        return action
    return program
```

L1–2	`q` maps a `(state, action)` pair to a learned value — the agent's policy, and the thing that changes over time. This is a miniature Q-table.
L3	Remembers the previous step, because the reward for an action only becomes visible on the *next* percept. This one-step delay is the central difficulty of reinforcement learning in miniature.
L5–6	The percept is used directly as the state.
L7	`NoOp` is deliberately excluded from the action set — the agent is never allowed to learn to idle.
L8–9	`setdefault` lazily initialises unseen pairs to 0.0, so line 21's lookup can never raise.
L11–12	The **learning element**, skipped on the very first step when there is no previous action to credit.
L13	The reward signal: `+10` if the square is now clean *and* the last action was `Suck`, otherwise `−1`. Note what this cannot distinguish — sucking dirt and sucking an already-clean square both look like success.
L14–15	The update: move the stored value *halfway* toward the observed reward. `0.5` is the learning rate — high, so it adapts within six steps, but it also means the value never settles.
L17–19	**Exploration.** 10% of the time, act randomly. Without this the agent would lock onto whatever it tried first and never discover anything better.
L20–21	**Exploitation.** The other 90%: take the highest-valued action for this state. Together, lines 18–21 are the classic epsilon-greedy policy.
L23–25	Record this step as “last” so the next call can score it, return the action, return the program.

WHY IT USES THE GLOBAL RANDOM

Lines 18–19 call `random` directly, not a seeded private generator. That is why the comparison cell has to call `random.seed(0)` before running — without it, this agent's score changes between runs and the table stops being reproducible. It is also why Exercise 1 re-seeds inside its loop.

SIX STEPS IS NOT LEARNING

It scores 10 here, but do not read that as the learning working. With six steps and a learning rate of 0.5 it barely fills its table. What this cell demonstrates is the *shape* of a learning agent — policy, reward, update, explore/exploit — not its competence.

09 Running all five side by side

Same world, same seed, same six steps. Every difference in the table is caused by architecture and nothing else.

Cell 11 — the comparison

```
def run(agent_program, steps=6, seed=42):
    env = VacuumEnvironment(seed=seed)
    for _ in range(steps):
        p = env.percept()
        a = agent_program(p)
        env.execute(a)
    return env.performance, env.is_clean(), env.history


random.seed(0)  # for reproducibility of the learning agent's exploration
agents = [
    ("Simple Reflex", simple_reflex_agent),
    ("Model-Based Reflex", model_based_reflex_agent),
    ("Goal-Based", goal_based_agent),
    ("Utility-Based", utility_based_agent),
    ("Learning", learning_agent),
]

print(f"{'Architecture':22s}{'Performance':>12s}{'All Clean?':>12s}")
print("-" * 46)
for name, factory in agents:
    perf, clean, _ = run(factory())
    print(f"{name:22s}{perf:12d}{str(clean):>12s}")
```

L1–7	`run` defined a second time, character for character identical to cell 4. Harmless — it just makes this cell runnable on its own — but worth noticing so you do not go hunting for a difference that is not there.
L10	Seed the *global* generator. This exists solely for the learning agent's exploration on line 18 of the previous cell; without it the last row of the table would wobble.
L11–17	A list of `(name, factory)` pairs. Storing the *factory* rather than a built program is what guarantees each agent starts with fresh internal state.
L19	The header row. `{'Architecture':22s}` left-aligns in 22 columns, `{'Performance':>12s}` right-aligns in 12 — that is what makes the columns line up.
L20	A rule of 46 dashes, matching 22+12+12.
L21–22	For each entry, call the factory to build a fresh program and run it. `_` discards the trace.
L23	Print the row. `{perf:12d}` formats the score as a right-aligned integer; `str(clean)` converts the bool before padding it, since `:>12s` requires a string.

OUTPUT

```
Architecture           Performance  All Clean?
---------------------------------------------
Simple Reflex                    5        True
Model-Based Reflex               9        True
Goal-Based                       5        True
Utility-Based                   10        True
Learning                        10        True
```

How to read this table

ARCHITECTURE	SCORE	WHAT ACTUALLY HAPPENED
Simple Reflex	5	Cleans on step 2, then paces for four steps at −1 each.
Model-Based Reflex	9	Same clean, then `NoOp` — it knows the job is done.
Goal-Based	5	Identical to simple reflex: no goal test, so no stopping.
Utility-Based	10	Starts on the dirty square, sucks, then never pays to move.
Learning	10	Explores into the same behaviour. Six steps is not enough to call this learned.

THE HONEST READING

The scores do **not** rank the architectures. Utility-based ties for first here by being lucky about where it started, goal-based ties for last because of a missing branch, and the learning agent's 10 is barely distinguishable from noise. A six-step, two-square, fully observable, episodic world is simply too easy to separate them.

That is exactly why the exercises exist. Each one breaks a different assumption in the PEAS spec, and each one produces a *different* ranking.

10 Exercise 1 — Dirt that comes back

The task: make dirt reappear at random after being cleaned, turning the world *stochastic* and *non-episodic*. Which architecture degrades least?

My hypothesis before running it: the agents that *stop* once they believe the job is done (model-based, goal-based) should suffer most, because their belief goes stale. The simple reflex agent, which never stops, should suffer least.

Solution — stochastic environment and the fix

```python
# Exercise 1 — Stochastic, non-episodic dirt
#
# Dirt can reappear after being cleaned, so the world never "ends".
# Hypothesis: the architectures that STOP once they believe the job is done
# (Model-Based, Goal-Based) degrade the most, because their belief goes stale.
# The Simple Reflex Agent, which never stops patrolling, degrades the least.

class StochasticVacuumEnvironment(VacuumEnvironment):
    """Same world, but each Clean square can get dirty again every time step."""

    def __init__(self, dirt_prob=0.5, respawn_prob=0.15, seed=42):
        super().__init__(dirt_prob=dirt_prob, seed=seed)
        self.respawn_prob = respawn_prob
        self.rng = random.Random(seed + 1000)   # separate stream, so respawns c
        self.steps = 0
        self.clean_square_steps = 0             # for measuring "how clean was t

    def execute(self, action):
        super().execute(action)
        for loc in LOCATIONS:                                  # the environment a
            if self.status[loc] == "Clean" and self.rng.random() < self.respawn_
                self.status[loc] = "Dirty"
        self.steps += 1
        self.clean_square_steps += sum(1 for v in self.status.values() if v == '
        return self.percept()

    def cleanliness(self):
        """Fraction of (square, time step) pairs that were clean. The right metr
        for a non-episodic task: there is no final state to score."""
        return self.clean_square_steps / (self.steps * len(LOCATIONS))


def run_stochastic(agent_program, steps=40, seed=42, respawn_prob=0.15):
    env = StochasticVacuumEnvironment(seed=seed, respawn_prob=respawn_prob)
    for _ in range(steps):
        env.execute(agent_program(env.percept()))
    return env.performance, env.cleanliness()


STEPS, SEEDS = 40, range(8)
stochastic_agents = [
    ("Simple Reflex",      simple_reflex_agent),
    ("Model-Based Reflex", model_based_reflex_agent),
    ("Goal-Based",         goal_based_agent),
```

```
    ("Utility-Based",      lambda: utility_based_agent(battery_start=STEPS)),  #
    ("Learning",           learning_agent),
]

print(f"{'Architecture':22s}{'Avg performance':>17s}{'Avg cleanliness':>17s}")
print("-" * 56)
for name, factory in stochastic_agents:
    results = []
    for s in SEEDS:
        random.seed(0)                      # keep the learning agent's explora
        results.append(run_stochastic(factory(), steps=STEPS, seed=s))
    perf = sum(r[0] for r in results) / len(results)
    clean = sum(r[1] for r in results) / len(results)
    print(f"{name:22s}{perf:17.1f}{clean:16.0%}")

print("""
Reading the table:

* Simple Reflex and Goal-Based tie at the top. Neither ever stops: the goal-
  based agent as written has no NoOp branch, so in this world it degenerates
  into the same patrol loop as the reflex agent. Constant re-sensing is
  exactly the right behaviour when dirt keeps coming back.
* Model-Based Reflex drops. It latches onto "both squares are Clean", emits
  NoOp forever, and never looks again - its model is a snapshot of a world
  that has since changed.
* Utility-Based drops furthest. Moving costs -2 while Suck-on-a-clean-square
  costs 0, so it never pays to walk to the other square. It parks on one
  square, cleans that one forever, and lets the other rot.
* Learning does no better: its reward signal only rewards Suck, so it learns
  the same park-and-suck policy.

So the hypothesis was half right. The thing that degrades is not "having a
model" - it is *trusting a stale model* (Model-Based) or *pricing movement
without pricing neglect* (Utility-Based). The fix for the first is to make
beliefs expire:""")


def decaying_model_based_agent(staleness_limit=3):
    """Model-based, but a belief older than `staleness_limit` steps is not trust
    model = {loc: None for loc in LOCATIONS}
    last_seen = {loc: -10**9 for loc in LOCATIONS}
    t = {"step": 0}

    def program(percept):
```

```
        location, status = percept
        t["step"] += 1
        model[location] = status
        last_seen[location] = t["step"]
        if status == "Dirty":
            return "Suck"
        believed_clean = all(
            model[loc] == "Clean" and t["step"] - last_seen[loc] <= staleness_li
            for loc in LOCATIONS
        )
        if believed_clean:
            return "NoOp"
        return "Right" if location == "A" else "Left"
    return program


results = [run_stochastic(decaying_model_based_agent(), steps=STEPS, seed=s) for
print(f"\n{'Model-Based + decay':22s}"
      f"{sum(r[0] for r in results)/len(results):17.1f}"
      f"{sum(r[1] for r in results)/len(results):16.0%}")
```

L1–6	The hypothesis, written down before the code runs. Worth doing: half of it turns out to be wrong, and that is the interesting part.
L8–9	Subclass rather than edit the original, so every earlier cell still works and the two worlds can be compared directly.
L11–12	`super().__init__` builds the ordinary starting world, so the only difference is what happens afterwards.
L13	How likely a clean square is to become dirty again, each step.
L14	A *separate* random stream, offset by 1000. Without this, respawn draws would consume numbers from the same generator that built the starting world, and changing `respawn_prob` would silently change the starting layout too — you could no longer tell which variable caused a difference.
L15–16	Counters for the new metric.
L18–19	Override `execute`; delegate the action's normal effects to the parent.
L20–22	**The change that matters.** After the agent acts, the environment acts on its own: each clean square may become dirty again. The world is no longer something that only changes when the agent touches it.
L23–24	Accumulate how many squares were clean at each step.
L25	Return the fresh percept, matching the parent's contract.
L27–30	**A new performance measure.** `is_clean()` is meaningless now — there is no final state, and whether the world happens to be clean on the last step is luck. Instead: what fraction of (square, time step) pairs were clean? Changing the environment forced changing the metric, which is the deeper lesson of this exercise.
L33–37	A runner for the new world, structured like the original but returning the new metric.
L40	40 steps and 8 seeds — long enough for respawns to matter, repeated enough to average out lucky worlds.
L41–47	The same five architectures. Line 45 wraps the utility agent in a `lambda` to give it a battery of 40 instead of 6 — otherwise it would simply run flat at step 6 and the comparison would measure battery size, not architecture.

L49–50	Header and rule, same formatting trick as the original comparison.
L51–55	For each agent, run every seed. `random.seed(0)` inside the loop keeps the learning agent reproducible.
L56–58	Average both metrics and print the row. `{clean:16.0%}` formats a fraction as a whole-number percentage.
L60–79	The written analysis, printed with the results so the conclusion never drifts from the numbers it describes.
L82–83	**The fix.** A model-based agent whose beliefs expire.
L84–86	Same model as before, plus a record of *when* each square was last observed, and a step counter. Line 85's `-10**9` is a sentinel meaning “never seen”, guaranteed to look infinitely stale.
L88–92	Advance the clock, record the observation and its timestamp.
L93–94	Unchanged reflex rule.
L95–98	**The key condition.** The agent now believes the world is clean only if every square is *both* believed clean *and* was observed within the last `staleness_limit` steps. Knowledge has an expiry date.
L99–101	Idle if that holds; otherwise patrol.
L105–108	Run the fixed agent on the same seeds and print its row under the table for comparison.

OUTPUT

```
Architecture            Avg performance  Avg cleanliness
--------------------------------------------------------
Simple Reflex                      93.4             73%
Model-Based Reflex                 67.8             52%
Goal-Based                         93.4             73%
Utility-Based                      58.8             45%
Learning                           58.9             45%

Reading the table:

* Simple Reflex and Goal-Based tie at the top. Neither ever stops: the goal-
  based agent as written has no NoOp branch, so in this world it degenerates
  into the same patrol loop as the reflex agent. Constant re-sensing is
  exactly the right behaviour when dirt keeps coming back.
* Model-Based Reflex drops. It latches onto "both squares are Clean", emits
  NoOp forever, and never looks again - its model is a snapshot of a world
  that has since changed.
* Utility-Based drops furthest. Moving costs -2 while Suck-on-a-clean-square
  costs 0, so it never pays to walk to the other square. It parks on one
  square, cleans that one forever, and lets the other rot.
* Learning does no better: its reward signal only rewards Suck, so it learns
  the same park-and-suck policy.

So the hypothesis was half right. The thing that degrades is not "having a
model" - it is *trusting a stale model* (Model-Based) or *pricing movement
without pricing neglect* (Utility-Based). The fix for the first is to make
beliefs expire:

Model-Based + decay               100.1             69%
```

What the numbers actually said

The hypothesis was **half right**, and being wrong about the other half is the most useful thing in this exercise.

ARCHITECTURE	PERF.	CLEAN	WHAT HAPPENED
Simple Reflex	93.4	73%	Never stops, so it re-senses constantly. Memorylessness is now an *advantage*.
Goal-Based	93.4	73%	**Did not collapse.** It has no `NoOp` branch, so it degenerates into the same patrol loop.
Model-Based	67.8	52%	Latches onto “both clean” and idles forever on a snapshot of a world that has moved on.
Utility-Based	58.8	45%	Worst. Moving scores −2, sucking a clean square scores 0 — so it parks on one square and lets the other rot.
Learning	58.9	45%	Its reward only credits `Suck`, so it learns the same park-and-suck policy.
Model-Based + decay	**100.1**	69%	Best performance of all: patrols like the reflex agent but skips pointless moves.

THE REAL ANSWER

What degrades is not *having a model*. It is **trusting a stale model** (model-based) or **pricing movement without pricing neglect** (utility-based). Note the fix does not remove the memory — it adds an expiry date to it, and comes out ahead of every original agent.

Also note the utility agent's bug was invisible in the original tutorial. Its 10/10 score there came from starting on the dirty square. This exercise is what exposed it.

11 Exercise 2 — Taking away the location sensor

The task: `percept()` returns only the dirt status, never the location. Which architectures still work, which break, and why?

Solution — three agents under partial observability

```python
# Exercise 2 — Partial observability: the agent can no longer see WHERE it is
#
# percept() now returns only the dirt status of the current square.
# Question: which architectures survive?

class BlindVacuumEnvironment(VacuumEnvironment):
    """Sensors are degraded: dirt status only, no location."""

    def percept(self):
        return self.status[self.agent_location]      # a bare "Dirty"/"Clean", no


def run_blind(agent_program, steps=6, seed=3):
    env = BlindVacuumEnvironment(seed=seed)
    for _ in range(steps):
        env.execute(agent_program(env.percept()))
    return env.performance, env.is_clean(), env.history


# --- (a) Simple reflex with a fixed rule: BREAKS ------------------------------
def blind_fixed_reflex_agent():
    """No memory, no location. The best it can do is one fixed move direction."""
    def program(status):
        return "Suck" if status == "Dirty" else "Right"
    return program


# --- (b) Simple reflex + randomisation: WORKS, eventually ---------------------
def blind_random_reflex_agent(seed=7):
    """AIMA's answer to unobservable location: randomise the move.
    Still memoryless, but no longer trapped in a corner."""
    rng = random.Random(seed)
    def program(status):
        return "Suck" if status == "Dirty" else rng.choice(["Left", "Right"])
    return program


# --- (c) Model-based: WORKS, and works perfectly ------------------------------
def blind_model_based_agent():
    """Key insight: the ACTUATORS are absolute and deterministic - "Left" always
    lands on A, "Right" always lands on B. So after the agent's first move it
    knows exactly where it is, even though the sensors never tell it.
    Internal state has *reconstructed* the observability the sensors lost."""
    belief = {"location": None, "A": None, "B": None}
```

```
    def program(status):
        loc = belief["location"]
        if loc is not None:
            belief[loc] = status                        # we know where this read
        if status == "Dirty":
            if loc is not None:
                belief[loc] = "Clean"                   # Suck will succeed
            return "Suck"
        if loc is None:                                 # first clean percept: lo
            belief["location"] = "A"
            return "Left"                               # "Left" is a teleport-to
        if belief["A"] == "Clean" and belief["B"] == "Clean":
            return "NoOp"
        belief["location"] = "B" if loc == "A" else "A"
        return "Right" if loc == "A" else "Left"
    return program


for label, factory in [("(a) fixed reflex  ", blind_fixed_reflex_agent),
                       ("(b) random reflex ", blind_random_reflex_agent),
                       ("(c) model-based   ", blind_model_based_agent)]:
    perf, clean, trace = run_blind(factory())
    actions = " ".join(a for _, a, _ in trace)
    print(f"{label} perf={perf:4d}  all clean={str(clean):5s}  actions: {actions}

print("""
* (a) Fixed reflex BREAKS. With no location in the percept it cannot choose a
  direction, so it commits to one - "Right" - forever. In this run it starts
  on a clean B and simply re-enters B six times, paying the move penalty each
  step while the dirt in A is never found (perf -6, world never clean). Note
  it does not fail loudly; it just quietly stops making progress.
* (b) Randomised reflex WORKS, in expectation. Still no memory, but a coin flip
  guarantees it visits both squares eventually. This is the classic result:
  when you cannot sense, randomise.
* (c) Model-based WORKS, and cleanly. Because Left/Right are absolute moves,
  one deliberate "Left" tells the agent it is now at A - it has localised
  itself using its own action history rather than its sensors. From there it
  tracks position by dead reckoning and can still emit NoOp when done.

The general rule: partial observability breaks memoryless agents, and internal
state is what buys the observability back. That is precisely why the
model-based box exists in the AIMA hierarchy.""")
```

L1–4	The setup: this is a *sensor* downgrade, breaking the S in PEAS.
L6–10	Subclass overriding one method. `percept` now returns a bare `"Dirty"` / `"Clean"` string instead of a tuple. That is the entire change — one line, and it invalidates most of the agents written so far.
L13–17	A runner for the blind world, otherwise identical to the original.
L21–25	**(a) The memoryless agent.** Its program now takes `status` directly, since there is no tuple to unpack. With no location it cannot choose a direction, so line 24 hard-codes one: suck dirt, otherwise always go right.
L29–35	**(b) Randomised.** Identical, except line 34 flips a coin instead of committing to a direction. Still completely memoryless — the `rng` is not state about the world, just a source of variety.
L39–43	**(c) Model-based — the interesting one.** The docstring holds the insight: the *actuators* are absolute and deterministic. `Left` always lands on A regardless of where you were. So the agent can learn its position from its own *actions*, even though its sensors will never tell it.
L44	Belief state: where I think I am, plus what I have seen at each square.
L46–49	If the agent knows where it is, it can attribute the reading to that square. If it does not, the reading is unattributable and must be discarded.
L50–53	Dirt gets sucked regardless of whether the agent knows where it is — that rule never needed a location. Line 52 pre-records the result, since `Suck` is guaranteed to succeed on a dirty square.
L54–56	**The self-localisation move.** Lost, on a clean square: deliberately go `Left`. Because that is a teleport-to-A, the agent *now knows* it is at A. It has spent one move to convert “I am lost” into “I am at A”.
L57–58	Once both squares are known clean, idle — a capability the blind reflex agents cannot express at all.
L59–60	Otherwise move to the other square, updating the belief about position *before* moving. This is dead reckoning: tracking position by remembering your own actions.

L64–69	Run all three and print each one's full action sequence, which is what makes the failure visible.
L71–87	The analysis, printed alongside.

OUTPUT

```
(a) fixed reflex   perf=  -6  all clean=False  actions: Right Right Right Right Right Rig
(b) random reflex  perf=   5  all clean=True   actions: Right Left Suck Right Left Left
(c) model-based    perf=   8  all clean=True   actions: Left Suck Right NoOp NoOp NoOp

* (a) Fixed reflex BREAKS. With no location in the percept it cannot choose a
  direction, so it commits to one - "Right" - forever. In this run it starts
  on a clean B and simply re-enters B six times, paying the move penalty each
  step while the dirt in A is never found (perf -6, world never clean). Note
  it does not fail loudly; it just quietly stops making progress.
* (b) Randomised reflex WORKS, in expectation. Still no memory, but a coin flip
  guarantees it visits both squares eventually. This is the classic result:
  when you cannot sense, randomise.
* (c) Model-based WORKS, and cleanly. Because Left/Right are absolute moves,
  one deliberate "Left" tells the agent it is now at A - it has localised
  itself using its own action history rather than its sensors. From there it
  tracks position by dead reckoning and can still emit NoOp when done.

The general rule: partial observability breaks memoryless agents, and internal
state is what buys the observability back. That is precisely why the
model-based box exists in the AIMA hierarchy.
```

The verdict

AGENT	RESULT	WHY
(a) Fixed reflex	**Breaks** (−6, never clean)	Committed to `Right`, started on a clean B, and re-entered B six times. The dirt in A was never found.
(b) Random reflex	Works, eventually (+5)	A coin flip guarantees it visits both squares given enough steps. When you cannot sense, randomise.
(c) Model-based	**Works perfectly** (+8)	`Left` → `Suck` → `Right` → `NoOp`. It localised itself, cleaned, verified, and stopped.

> THE MOST IMPORTANT DETAIL

Agent (a) does not crash. It raises nothing, logs nothing, and returns a legal action every single step. It just quietly stops making progress while paying a penalty. Degraded sensors produce *silent* failures, which is precisely why you cannot catch this class of bug by checking that the code runs.

THE GENERAL RULE

Partial observability breaks memoryless agents, and internal state is what buys the observability back. Agent (c) never gains a location sensor — it *reconstructs* location from its own action history. That is the whole reason the model-based box exists in the hierarchy.

12 Exercise 3 — Scaling to a row of four

The task: extend `LOCATIONS` to `["A","B","C","D"]` in a row, update the movement logic, and ask whether the simple reflex agent still reaches `NoOp` sensibly.

SCALING FORCES A SEMANTIC CHANGE

With two squares, `Left` could be *absolute* — "go left" and "go to A" were the same instruction. In a row of four they are not. Movement has to become *relative*, and that single change is what makes this exercise more than bookkeeping. (It is also what Exercise 2's agent (c) was quietly exploiting.)

Solution — an n-square row

```python
# Exercise 3 — Scale up to a row of four squares: A - B - C - D
#
# Note the change of semantics: with two squares "Left"/"Right" could be
# absolute (teleport to A / teleport to B). In a row of four they must become
# RELATIVE moves, which is what makes the exercise interesting.

ROW = ["A", "B", "C", "D"]


class RowVacuumEnvironment:
    """Same PEAS, but n squares in a row and Left/Right are one step, clamped at

    def __init__(self, locations=ROW, dirt_prob=0.5, seed=42):
        rng = random.Random(seed)
        self.locations = list(locations)
        self.status = {loc: ("Dirty" if rng.random() < dirt_prob else "Clean") f
        self.index = rng.randrange(len(self.locations))
        self.performance = 0
        self.history = []

    @property
    def agent_location(self):
        return self.locations[self.index]

    def percept(self):
        return (self.agent_location, self.status[self.agent_location])

    def execute(self, action):
        loc = self.agent_location
        if action == "Suck":
            if self.status[loc] == "Dirty":
                self.status[loc] = "Clean"
                self.performance += 10
        elif action == "Left":
            self.index = max(0, self.index - 1)          # clamp: bumping into t
            self.performance -= 1
        elif action == "Right":
            self.index = min(len(self.locations) - 1, self.index + 1)
            self.performance -= 1
        elif action == "NoOp":
            pass
        else:
            raise ValueError(f"Unknown action: {action}")
        self.history.append((loc, action, dict(self.status)))
```

```
        return self.percept()

    def is_clean(self):
        return all(v == "Clean" for v in self.status.values())


def run_row(agent_program, steps=14, seed=42):
    env = RowVacuumEnvironment(seed=seed)
    for _ in range(steps):
        env.execute(agent_program(env.percept()))
    return env.performance, env.is_clean(), env.history


def row_simple_reflex_agent(locations=ROW):
    """Memoryless. The only thing it can key off is (location, status), so the
    best fixed rule is: sweep right, and at the right wall sweep left."""
    def program(percept):
        location, status = percept
        if status == "Dirty":
            return "Suck"
        return "Left" if location == locations[-1] else "Right"
    return program


def row_model_based_agent(locations=ROW):
    """Internal state: what we have seen at every square. Sweep right to the wal
    then left, and stop once all four are believed clean."""
    model = {loc: None for loc in locations}
    direction = {"d": "Right"}

    def program(percept):
        location, status = percept
        model[location] = status
        if status == "Dirty":
            model[location] = "Clean"
            return "Suck"
        if all(v == "Clean" for v in model.values()):
            return "NoOp"                                  # every square seen,
        if location == locations[-1]:
            direction["d"] = "Left"
        elif location == locations[0]:
            direction["d"] = "Right"
        return direction["d"]
    return program
```

```
89
90
91  for label, factory in [("Simple Reflex     ", row_simple_reflex_agent),
92                         ("Model-Based Reflex", row_model_based_agent)]:
93      perf, clean, trace = run_row(factory())
94      actions = " ".join(a for _, a, _ in trace)
95      print(f"{label}  perf={perf:4d}  all clean={str(clean):5s}")
96      print(f"{'':20s}actions: {actions}")
97
98  print("""
99  Does the Simple Reflex Agent still reach NoOp sensibly? No - and it is worse
100 than that. Look at its action trace: after cleaning its way right it settles
101 into "Left Right Left Right ..." - it is not patrolling the row, it is stuck
102 oscillating between the last two squares.
103
104 Why: the rule "Left at the right wall, otherwise Right" makes the right wall a
105 2-cycle. Direction of travel is *state*, and a memoryless agent has nowhere to
106 keep it, so it cannot commit to a leftward sweep. With n=4 it gets away with
107 this, because it cleaned A-D on the way out. With n=8 it does not - it strands
108 itself at the far end while dirt sits behind it:""")
109
110 for n in (2, 4, 8):
111     locations = [chr(ord("A") + i) for i in range(n)]
112     for label, factory in [("Simple Reflex", row_simple_reflex_agent),
113                            ("Model-Based ", row_model_based_agent)]:
114         env = RowVacuumEnvironment(locations=locations, seed=42)
115         program = factory(locations)
116         for _ in range(4 * n):
117             env.execute(program(env.percept()))
118         print(f"  n={n}  {label}  perf={env.performance:4d}  clean={env.is_clear
119
120 print("""
121 And NoOp is out of reach for a second, independent reason: NoOp is only
122 rational once you know *every* square is clean, and "every square" is a fact
123 about squares you are not standing on. A memoryless agent has no place to put
124 that fact, so its rule set cannot even express the NoOp condition. The
125 model-based agent keeps both pieces of state - direction of travel and what it
126 has seen where - so it sweeps properly and then stops.
127
128 Exercise 2's lesson applies here too: if you must stay memoryless, randomise."""
129
130
131 def row_random_reflex_agent(locations=ROW, seed=7):
132     """Memoryless, but breaks the oscillation trap with a coin flip."""
```

```
    rng = random.Random(seed)
    def program(percept):
        _, status = percept
        return "Suck" if status == "Dirty" else rng.choice(["Left", "Right"])
    return program


for n in (4, 8):
    locations = [chr(ord("A") + i) for i in range(n)]
    cleaned, perfs = 0, []
    for trial in range(20):
        env = RowVacuumEnvironment(locations=locations, seed=42)
        program = row_random_reflex_agent(locations, seed=trial)
        for _ in range(4 * n):
            env.execute(program(env.percept()))
        cleaned += env.is_clean()
        perfs.append(env.performance)
    print(f"  n={n}  Random Reflex over {4*n} steps: cleaned the row in "
          f"{cleaned}/20 trials, avg perf {sum(perfs)/len(perfs):.1f}")

print("""
Randomising escapes the trap but is slow: a random walk's cover time grows
like n-squared, so within a fixed step budget it often has not visited
everything yet. Memorylessness is survivable, not free - which is the point of
the exercise.""")
```

L1–7	The setup and the new world: four squares in a row.
L10–11	A fresh class rather than a subclass — the movement model differs too much to inherit cleanly.
L13–16	As before, but the world size is now a parameter. Line 16's comprehension is unchanged except that it iterates over `self.locations`.
L17	Position is now an **index**, not a name. That is the core representational change: in a row you need to know how far along you are, not just which label you are on.
L18–19	Scorecard and trace, unchanged.
L21–23	A `@property` so `agent_location` still reads like an attribute, keeping every agent's `percept` unpacking working unchanged. The old interface is preserved over a new representation.
L25–26	Identical percept contract: current location and its status.
L28–33	`Suck` is unchanged — it never depended on the world's shape.
L34–36	**`Left` is now relative:** one step back, `max(0, ...)` clamping at the wall. Note the −1 is charged *even when the move is blocked* — walking into a wall costs you.
L37–39	`Right` mirrors it, clamped at the far end.
L40–43	`NoOp` and the error branch, unchanged.
L44–45	`dict(self.status)` is a shallow copy — sufficient here, because the values are immutable strings.
L47–55	`is_clean` and the runner, unchanged in structure.
L58–66	**The reflex agent, scaled.** Memoryless, so its rule can only key off `(location, status)`. Line 65 is the best fixed rule available: sweep right, and reverse at the right wall.
L69–73	**The model-based agent, scaled.** Two pieces of state now: what it has seen at every square (line 72) *and* which way it is travelling (line 73). That second one is new — it did not need it with two squares.

L75–80	Observe, then suck dirt. Line 79 pre-records the success.
L81–82	Idle once every square is known clean. `all(...)` over the model is `False` while any entry is still `None`, so the agent cannot stop before visiting everywhere — the unvisited state is doing real work.
L83–87	Reverse direction at either wall, then travel that way. This is the sweep the reflex agent cannot perform.
L91–96	Run both and print their action sequences.
L98–108	The analysis of what those sequences show.
L110–118	**The scaling test.** Same agents at n=2, 4 and 8, with a step budget of `4n` so the budget scales with the problem. Line 111 generates location names with `chr(ord('A') + i)`.
L120–128	The second half of the analysis: the independent reason `NoOp` is unreachable.
L131–137	A randomised memoryless agent, applying Exercise 2's lesson to this problem.
L140–151	20 trials per size, since a random agent's single run tells you nothing. Line 148 exploits `True == 1` to count successes.
L153–157	The closing caveat about how slow random search actually is.

OUTPUT

```
Simple Reflex       perf=  19  all clean=True
                    actions: Right Suck Right Suck Right Suck Left Right Left Right Left
Model-Based Reflex  perf=  27  all clean=True
                    actions: Right Suck Right Suck Right Suck NoOp NoOp NoOp NoOp NoOp No

Does the Simple Reflex Agent still reach NoOp sensibly? No - and it is worse
than that. Look at its action trace: after cleaning its way right it settles
into "Left Right Left Right ..." - it is not patrolling the row, it is stuck
oscillating between the last two squares.

Why: the rule "Left at the right wall, otherwise Right" makes the right wall a
2-cycle. Direction of travel is *state*, and a memoryless agent has nowhere to
keep it, so it cannot commit to a leftward sweep. With n=4 it gets away with
this, because it cleaned A-D on the way out. With n=8 it does not - it strands
itself at the far end while dirt sits behind it:
  n=2  Simple Reflex  perf=   3  clean=True
  n=2  Model-Based    perf=   9  clean=True
  n=4  Simple Reflex  perf=  17  clean=True
  n=4  Model-Based    perf=  27  clean=True
  n=8  Simple Reflex  perf= -21  clean=False
  n=8  Model-Based    perf=  32  clean=True

And NoOp is out of reach for a second, independent reason: NoOp is only
rational once you know *every* square is clean, and "every square" is a fact
about squares you are not standing on. A memoryless agent has no place to put
that fact, so its rule set cannot even express the NoOp condition. The
model-based agent keeps both pieces of state - direction of travel and what it
has seen where - so it sweeps properly and then stops.

Exercise 2's lesson applies here too: if you must stay memoryless, randomise.
  n=4  Random Reflex over 16 steps: cleaned the row in 14/20 trials, avg perf 13.7
  n=8  Random Reflex over 32 steps: cleaned the row in 13/20 trials, avg perf 4.8

Randomising escapes the trap but is slow: a random walk's cover time grows
like n-squared, so within a fixed step budget it often has not visited
everything yet. Memorylessness is survivable, not free - which is the point of
the exercise.
```

The answer: no, and worse than you would expect

Look at the reflex agent's trace: after cleaning its way right it settles into `Left Right Left Right ...`. It is not patrolling the row — it is **stuck oscillating between the last two squares**.

The rule "`Left` at the right wall, otherwise `Right`" makes that wall a 2-cycle: step left off the wall, and the rule immediately sends you back. **Direction of travel is state**, and a memoryless agent has nowhere to keep it, so it cannot commit to a leftward sweep. At n=4 it gets away with this because it cleaned everything on the way out. At n=8 it does not:

ROW SIZE	SIMPLE REFLEX	MODEL-BASED	RANDOM REFLEX (20 TRIALS)
n = 2	3, clean	9, clean	—
n = 4	17, clean	27, clean	cleaned 14/20, avg 13.7
n = 8	**−21, never clean**	32, clean	cleaned 13/20, avg 4.8

And `NoOp` is out of reach for a second, independent reason: it is only rational once you know *every* square is clean — a fact about squares you are not standing on. A memoryless agent has no place to put that fact, so its rule set cannot even express the stopping condition.

> RANDOMISING IS SURVIVABLE, NOT FREE
>
> The coin flip escapes the oscillation trap, but a random walk's cover time grows like n^2, so inside a fixed step budget it frequently has not visited everything yet — it fails to clean the row in about a third of trials at both sizes. Memory is not a luxury here; it is the difference between $O(n)$ and $O(n^2)$.

13 Exercise 4 — Adding time pressure

The task: add a time-pressure term to `utility_based_agent` — a deadline after which `Suck` is worth less — and observe how the agent's choices shift.

Two supporting repairs are needed first, both diagnosed in Exercise 1. The original utility agent prices a move at a flat −2 with no term for what it might find, so **travel is never worth it**; and its beliefs never expire, so a square it cleaned long ago still looks clean. Adding a deadline to an agent that refuses to move would demonstrate nothing.

Solution — deadline, lookahead and decaying belief

```python
# Exercise 4 — Add time pressure to the utility function
#
# Three changes to `utility_based_agent`:
#   1. A DEADLINE term: a clean square is worth full value up to the deadline
#      and decays afterwards ("the shift is ending, cleaning matters less").
#   2. A one-step LOOKAHEAD, so moving can be worth something. In the original,
#      moving scored a flat -2 while Suck-on-a-clean-square scored 0, so the
#      agent could never justify walking anywhere - the parking bug Exercise 1
#      exposed.
#   3. A DECAYING BELIEF, so a square we cleaned long ago looks suspicious
#      again - Exercise 1's staleness fix, expressed as a probability instead
#      of a hard cutoff.
#
# Runs in Exercise 1's StochasticVacuumEnvironment, so run that cell first.

def utility_agent_with_deadline(battery_start=6, deadline=4, decay=0.6,
                                respawn_belief=0.15, verbose=False):
    state = {"battery": battery_start, "t": 0}
    belief = {loc: "Unknown" for loc in LOCATIONS}
    last_seen = {loc: 0 for loc in LOCATIONS}

    def time_factor(t):
        """What a clean square is worth at time t: full value up to the deadline
        return 1.0 if t <= deadline else decay ** (t - deadline)

    def p_dirty(dest, t):
        """How likely is the square over there to be dirty, given what we last s
        if belief[dest] == "Dirty":
            return 1.0
        if belief[dest] == "Unknown":
            return 0.5
        age = t - last_seen[dest]                              # the longer ago w
        return 1 - (1 - respawn_belief) ** age

    def utility(action, location, status, t, battery):
        if action == "Suck":
            return 10 * time_factor(t) if status == "Dirty" else -0.5   # suckin
        if action in ("Left", "Right"):
            dest = "A" if action == "Left" else "B"
            if dest == location:
                return -5.0                                    # a move that goes
            u = p_dirty(dest, t) * 10 * time_factor(t + 1) - 2
            if battery <= 2:
                u -= 5                                         # hoard energy whe
```

```
            return u
        return 0.0                                          # NoOp: the do-not

    def program(percept):
        location, status = percept
        state["t"] += 1
        belief[location] = status
        last_seen[location] = state["t"]
        if state["battery"] <= 0:
            return "NoOp"
        scores = {a: utility(a, location, status, state["t"], state["battery"])
                  for a in ("Suck", "Left", "Right", "NoOp")}
        action = max(scores, key=scores.get)
        if verbose:
            pretty = "  ".join(f"{a}={scores[a]:+6.1f}" for a in ("Suck", "Left"
            flag = "" if state["t"] <= deadline else "  (past deadline)"
            print(f"  t={state['t']:2d} at {location} ({status:5s})  {pretty}  -
        if action == "Suck":
            belief[location] = "Clean"
        if action in ("Left", "Right", "Suck"):
            state["battery"] -= 1
        return action
    return program


STEPS = 18
print(f"One run, deadline=6 — watch the agent give up as the clock runs out:\n")
env = StochasticVacuumEnvironment(seed=5, respawn_prob=0.15)
program = utility_agent_with_deadline(battery_start=STEPS, deadline=6, verbose=T
for _ in range(STEPS):
    env.execute(program(env.percept()))
print(f"\n  -> performance={env.performance}, cleanliness={env.cleanliness():.0%

print(f"{'deadline':>10s}{'avg performance':>18s}{'avg cleanliness':>18s}")
print("-" * 46)
for deadline in (STEPS, 14, 12, 10, 8, 6, 4, 2, 0):
    results = []
    for s in range(60):
        env = StochasticVacuumEnvironment(seed=s, respawn_prob=0.15)
        program = utility_agent_with_deadline(battery_start=STEPS, deadline=dead
        for _ in range(STEPS):
            env.execute(program(env.percept()))
        results.append((env.performance, env.cleanliness()))
    print(f"{deadline:>10d}{sum(r[0] for r in results)/len(results):>18.1f}"
```

```
          f"{sum(r[1] for r in results)/len(results):>17.0%}")

print("""
How the choices shift: early in the run a trip to the other square is worth
roughly p_dirty x 10 minus the -2 travel cost, so the agent patrols and cleans.
Once t passes the deadline, time_factor shrinks that payoff geometrically while
the travel cost stays at -2. The move term crosses below NoOp's flat 0 and the
agent stops travelling; a little later even Suck-on-dirt falls under 0 and it
stops cleaning altogether. Tighten the deadline and that crossover simply
happens sooner, so the table slides down gradually instead of flipping at one
value - cleanliness falls monotonically from 76% to 50%, and performance
tracks it (the 6-vs-4 blip is seed noise, not a real reversal).

That is what makes utility agents worth the extra machinery: nobody wrote
"if late, stop cleaning". The behaviour falls out of arithmetic between
competing terms. It is also why they are the hardest to debug - Exercise 1's
failure (parking on one square forever) was not a broken rule, it was two
terms priced wrong.""")
```

L1–14	The three changes, and a note that this cell reuses Exercise 1's stochastic world — a static world has no time dimension worth pressuring.
L16–17	Every knob is a parameter, so the behaviour can be swept rather than argued about.
L18–20	State: battery and clock together, beliefs about each square, and when each was last observed. Squares start `"Unknown"`, which is distinct from clean or dirty.
L22–24	**The deadline term.** A clean square is worth full value up to the deadline; afterwards its worth decays geometrically at `decay ** (t - deadline)`. At `decay=0.6` that is a fast fade — five steps past the deadline, value is down to about 8%.
L26–33	**The decaying belief.** Known dirty is certainty (1.0); never visited is a coin flip (0.5); otherwise the probability it has become dirty again grows with how long ago you looked: `1 - (1 - respawn_belief) ** age`. This is Exercise 1's staleness fix expressed as a probability rather than a hard cutoff — and it is *calibrated to the true respawn rate*, which is what makes it a model rather than a guess.
L35–37	`Suck` on dirt is worth 10, scaled by the deadline. On a clean square it now scores **−0.5** rather than 0 — a small explicit penalty that breaks the original's tie with `NoOp` and stops the agent burning battery on pointless sucking.
L38–42	**The lookahead.** A move is worth the probability of finding dirt, times its deadline-scaled value, minus the travel cost. Line 40 catches a move to the square you are already on. This is the line that makes travel rational — and it is only possible because line 26 gives the agent an opinion about a square it cannot see.
L43–45	The low-battery penalty, carried over unchanged.
L46	`NoOp` is still exactly 0 — the fixed baseline every other option is measured against. This is what makes the crossover meaningful.
L48–52	Advance the clock, record the observation and its timestamp.
L53–54	The hard battery constraint still overrides everything.
L55–57	Score all four actions into a dict, then take the best. `max(scores, key=scores.get)` returns the winning *key*.
L58–61	Optional per-step tracing of every score. This is how you debug a utility agent: you cannot tell why it chose something without seeing the numbers it rejected.

`L62–66`	Update belief after sucking, drain battery for real actions, return.
`L70–76`	One verbose run: 18 steps, deadline at 6, so you can watch the transition happen live.
`L78–89`	The sweep: nine deadline settings × 60 seeds each. Sixty is not decoration — at 8 seeds the ordering was noisy enough to be misread as a real reversal.
`L91–106`	The analysis, including an explicit note that the small 6-versus-4 wobble is seed noise, not a finding.

OUTPUT

```
One run, deadline=6 — watch the agent give up as the clock runs out:

  t= 1 at A (Clean)  Suck=  -0.5  Left=  -5.0  Right=  +3.0  NoOp=  +0.0  -> Right
  t= 2 at B (Clean)  Suck=  -0.5  Left=  -0.5  Right=  -5.0  NoOp=  +0.0  -> NoOp
  t= 3 at B (Clean)  Suck=  -0.5  Left=  +0.8  Right=  -5.0  NoOp=  +0.0  -> Left
  t= 4 at A (Clean)  Suck=  -0.5  Left=  -5.0  Right=  -0.5  NoOp=  +0.0  -> NoOp
  t= 5 at A (Clean)  Suck=  -0.5  Left=  -5.0  Right=  +0.8  NoOp=  +0.0  -> Right
  t= 6 at B (Clean)  Suck=  -0.5  Left=  -1.1  Right=  -5.0  NoOp=  +0.0  -> NoOp
  t= 7 at B (Clean)  Suck=  -0.5  Left=  -1.0  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t= 8 at B (Clean)  Suck=  -0.5  Left=  -1.2  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t= 9 at B (Dirty)  Suck=  +2.2  Left=  -1.4  Right=  -5.0  NoOp=  +0.0  -> Suck  (past
  t=10 at B (Clean)  Suck=  -0.5  Left=  -1.6  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=11 at B (Clean)  Suck=  -0.5  Left=  -1.7  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=12 at B (Clean)  Suck=  -0.5  Left=  -1.8  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=13 at B (Dirty)  Suck=  +0.3  Left=  -1.9  Right=  -5.0  NoOp=  +0.0  -> Suck  (past
  t=14 at B (Clean)  Suck=  -0.5  Left=  -1.9  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=15 at B (Clean)  Suck=  -0.5  Left=  -2.0  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=16 at B (Clean)  Suck=  -0.5  Left=  -2.0  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=17 at B (Clean)  Suck=  -0.5  Left=  -2.0  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past
  t=18 at B (Clean)  Suck=  -0.5  Left=  -2.0  Right=  -5.0  NoOp=  +0.0  -> NoOp  (past

  -> performance=17, cleanliness=69%

  deadline   avg performance   avg cleanliness
-----------------------------------------------
        18              43.6              76%
        14              41.6              74%
        12              40.8              71%
        10              38.2              67%
         8              36.1              64%
         6              34.6              61%
         4              34.8              60%
         2              33.7              57%
         0              29.0              50%

How the choices shift: early in the run a trip to the other square is worth
roughly p_dirty x 10 minus the -2 travel cost, so the agent patrols and cleans.
Once t passes the deadline, time_factor shrinks that payoff geometrically while
the travel cost stays at -2. The move term crosses below NoOp's flat 0 and the
agent stops travelling; a little later even Suck-on-dirt falls under 0 and it
stops cleaning altogether. Tighten the deadline and that crossover simply
happens sooner, so the table slides down gradually instead of flipping at one
value - cleanliness falls monotonically from 76% to 50%, and performance
tracks it (the 6-vs-4 blip is seed noise, not a real reversal).

That is what makes utility agents worth the extra machinery: nobody wrote
"if late, stop cleaning". The behaviour falls out of arithmetic between
```

```
competing terms. It is also why they are the hardest to debug - Exercise 1's
failure (parking on one square forever) was not a broken rule, it was two
terms priced wrong.
```

What the trace shows

Follow the `Suck` column down the verbose run — the agent's own valuation of cleaning, decaying in real time:

STEP	VALUE OF SUCK ON DIRT	WHAT THE AGENT DOES
t ≤ 6 (before deadline)	`+10.0`	Patrols: `Right`, `Left`, `Right` — travel is clearly worth it
t = 9	`+2.2`	Still sucks dirt under it, but no longer travels for it
t = 13	`+0.3`	Barely worth the actuation
t ≥ 14	—	Idles. `NoOp`'s flat 0 now beats everything

The move column tells the same story from the other side: `Left` starts at `+3.0` and sinks to `−2.0`. Once it crosses below `NoOp`'s zero, travelling stops. A little later `Suck` crosses too, and cleaning stops.

The sweep

Tighten the deadline and the crossover simply happens earlier, so the table slides rather than flipping at some threshold — cleanliness falls monotonically from **76%** at no pressure to **50%** at maximum pressure.

> WHY THIS IS THE PAYOFF EXERCISE
>
> **Nobody wrote “if late, stop cleaning.”** There is no such branch anywhere in the code. The behaviour emerges from arithmetic between competing terms — which is exactly what utility-based agents are for, and exactly why they are the hardest to debug. Exercise 1's failure was not a broken rule; it was two terms priced wrong. A reflex agent's bugs are in its rules, where you can read them. A utility agent's bugs are in its *numbers*.

14 Where this lands

Across four exercises the same pattern keeps surfacing: **an architecture is not better or worse in the abstract — it is better or worse for an environment.**

ENVIRONMENT PROPERTY	WHAT IT BREAKS	WHAT IT REWARDS
Dirt reappears (non-episodic)	Beliefs that never expire	Re-sensing, or beliefs with a decay policy
Location not observable	Memoryless rules	Internal state — or randomisation as a cheap substitute
More than two squares	Fixed direction rules (the oscillation trap)	State for *direction*, not just contents
A deadline	Fixed goals	A utility function that can trade goals off

The through-line

Notice that every exercise found a bug the original six-step, two-square, fully observable episode could not surface:

- The **goal-based agent has no goal test** — hidden in the original because scoring 5 looked like a reasonable result rather than a missing branch.
- The **utility agent never travels** — hidden because it happened to start on the dirty square and scored top marks.
- The **model-based agent trusts stale beliefs** — hidden because in an episodic world beliefs cannot go stale.
- The **reflex agent oscillates at walls** — hidden because with two squares there is no wall to get stuck against.

The agents did not change. The environment got honest.

> CARRY THIS INTO LECTURE 2
>
> **Most agent failures are environment assumptions that stopped holding.** Not bad algorithms — correct algorithms, still running correctly, in a world that no longer matches the PEAS spec they were written against.

> That is why serious agent work is dominated by *evaluation* rather than by architecture, and it is the same reason LLM agents are hard: the environment is the open world, and it never stops drifting away from the spec you built for.

If you want to go further

1. Combine Exercises 1 and 3: dirt that respawns in a row of eight. Does the decaying model-based agent still hold up, or does the row get too long to keep beliefs fresh?
2. Give the learning agent a reward that credits cleanliness rather than the `Suck` action, and see whether it stops parking.
3. Make movement stochastic — a 10% chance a move fails — and watch dead reckoning break. That is the step from a model to a *probabilistic* model, and it is where the next part of the course goes.

Every code block on this page was executed in order in a single Python 3.11 kernel, and every output block is the real stdout from that run — nothing here is transcribed by hand. To save this as a PDF, use your browser's Print command; the page has a print stylesheet that drops the sidebar and keeps code blocks from splitting across pages.